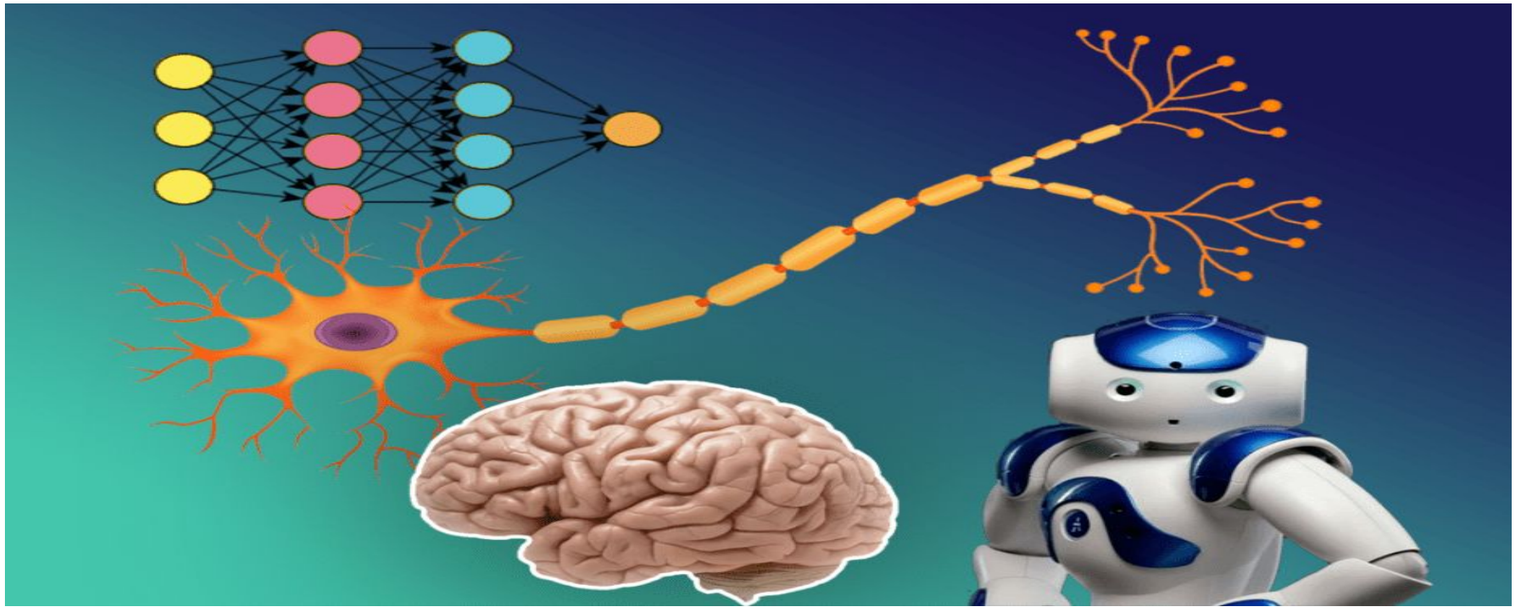
UNIT-I

Biological Inspiration of Neural Network



Topics :

- Introduction.
- Biological Inspiration of Neural Network
- Mathematical Neuron Model- Artificial Neural Network
- Components of ANN
- Artificial Neural Network-Perceptron
  - Perceptron Diagram
  - Perceptron Learning Algorithm
  - Representation of Power of Perceptron

Topics :

- Perceptron Learning Rule- Algorithm, Flowchart
- Layers in ANN
- Activation Function

Biological Inspiration of Neural Network

1. **Biological Neural Network (Human Brain):** The **human brain** is a highly complex information-processing system consisting of approximately **86 billion neurons** interconnected through trillions of synapses.

- A **typical brain** contains something like **100 billion microscopic cells** called **neurons.** **Each neuron** is made up of a **cell body (the central mass of the cell).** ie, the **sum of all inputs.**
- **Neurons are responsible for everything** that we do. Our **conscious & subconscious** is **controlled by these small power stations.**

Example

When a person sees a traffic signal:

1. **The eyes capture the image.** 2. **The brain processes the information.** The person recognizes the signal and decides whether to stop or move.

2. Observe the structure of a biological neuron: A biological neuron is the basic functional unit of the nervous system.

Components of a Biological Neuron

(a) Dendrites

- Receive signals from neighboring neurons.
- Carry information toward the cell body.

(b) Cell Body (Soma)

- Processes the received signals.
- Determines whether the signal strength is sufficient to generate an impulse.

(c) Axon

Carries electrical impulses away from the cell body.

- Transmits information to other neurons.

(d) Synapse

Junction between two neurons.

- Transfers information using chemical messengers called neurotransmitters.

BIOLOGICAL NEURON (NERVE CELL)

A. OVERVIEW DIAGRAM

Shows the complete flow of signal from one neuron to the next.

1 DENDRITES
Receive signals from other neurons.

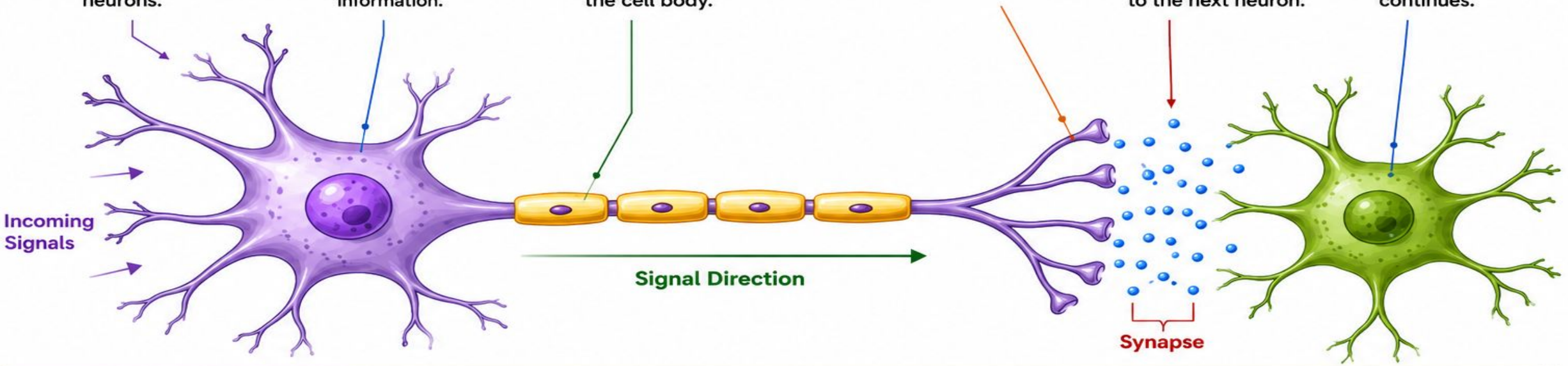
2 CELL BODY (SOMA)
Processes information.

3 AXON
Sends electrical signals away from the cell body.

4 AXON TERMINALS
Release chemical signals.

5 SYNAPSE
Transfer of signal (neurotransmitters) to the next neuron.

6 NEXT NEURON
Receives signal and the process continues.



B. FUNCTIONAL SUMMARY

Explains the function of each part in simple terms.

1 DENDRITES



- Receive signals
- Collect information from many sources

2 CELL BODY (SOMA)



- Integrates all inputs
- Decides whether to send signal

3 AXON



- Carries electrical impulses
- Sends signal away from cell body

4 AXON TERMINALS



- Release chemical signals
- Communicate with other neurons

5 SYNAPSE



- Gap between neurons
- Neurotransmitters cross the gap
- Signal is transferred

6 NEXT NEURON



- Receives the signal
- Further processing continues

Key Takeaway: Neurons work together in a network. Signals travel from dendrites → cell body → axon → axon terminals → synapse → next neuron.

3. Information flow in a biological neuron: The working of a biological neuron can be summarized as follows:

Stimulus → Dendrites → Soma → Axon → Synapse → Next Neuron

Working Procedure:

1. **Dendrites receive incoming signals.**
2. **The soma processes the signals.**
3. **If the signal exceeds a threshold, an electrical impulse is generated.**
4. **The impulse travels through the axon.**
5. **At the synapse, neurotransmitters transmit the signal to the next neuron.**

Mathematical Neuron Model- Artificial Neural Network

- Scientists converted the biological neuron into a mathematical model called the **Artificial Neuron.** An **Artificial Neural Network** consists of **many artificial neurons connected together in layers**.
- An **Artificial Neural Network(ANN)** may be defined as an **information – processing model** that is inspired by the **Biological Nervous System** , such as **Brain.**
- The model tries to **replicate only** the most **basic functions of the Brain.**
- An **ANN** is composed of a **large no. of highly interconnected processing units (Neurons)** working to solve the **specific Problem.**
- Like **Human Brain, ANN** is also a **Learn by Example**.
- Ie, Just as we **use our brains** to **identify patterns** and **classify different types of information**

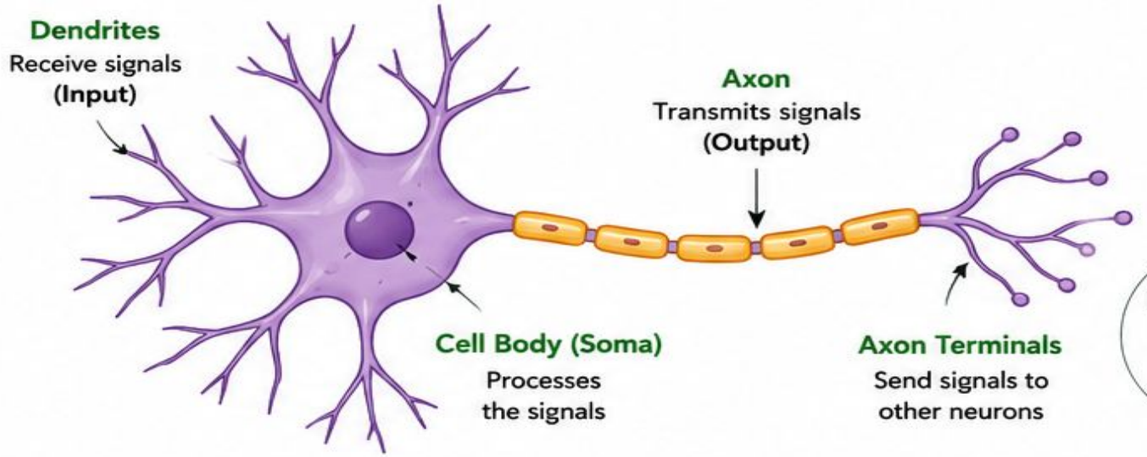
- By **using the Neural Networks**, it allowing **computers to observe, learn, and react to complex situations faster than humans.**
- An **ANN is configured** as **different types of Applications**, such as **Face Recognition, Pattern Recognition etc**. through a **learning process.**



Biological Inspiration for Neural Networks

Neural Networks are inspired by the structure and working of the human brain.

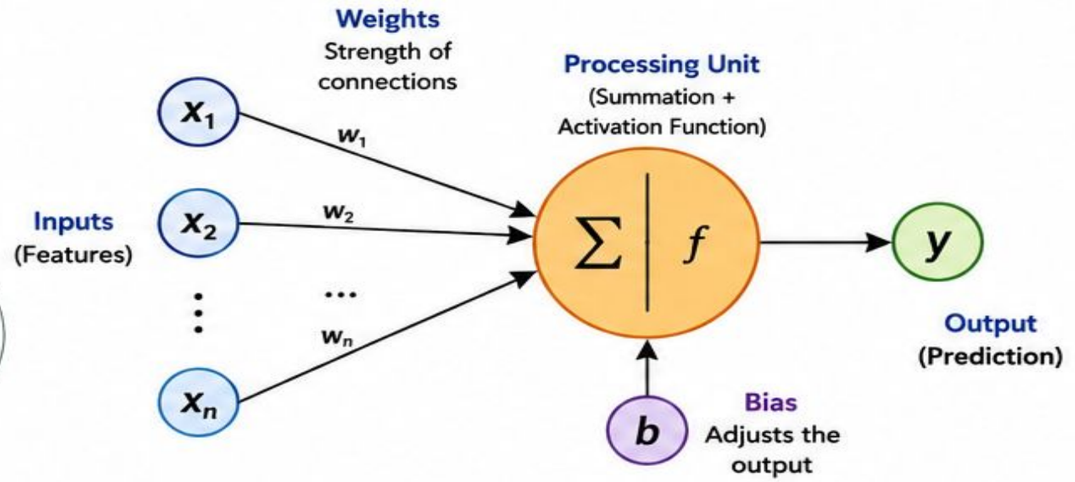
Human Brain (Biological Neuron)



Many dendrites receive signals → Cell body processes them → Axon sends the output → Terminals pass it to next neurons

INSPIRED BY

Artificial Neural Network (ANN Neuron)



Inputs (features) → Weights multiply inputs → Summation → Activation function → Output (prediction)

Comparison		
Biological Neuron	**Function**	**Artificial Neuron (ANN)**
Dendrites	Receive/Input	Input Features (x_1, x_2, …, x_n)
Synapses	Connection Strength	Weights (w_1, w_2, …, w_n)
Cell Body (Soma)	Process Information	Summation Function (Σ)
Activation of Neuron	Decision to Fire	Activation Function (f)
Axon	Transmit Output	Output (y)
Axon Terminals	Send to Next Neurons	Output to Next Layer
Learning from Experience	Adapt and Improve	Learning from Data (Training)

Key Takeaway

- The human brain has billions of neurons working together.
- Artificial Neural Networks (ANNs) are mathematical models inspired by biological neurons.
- They learn patterns from data and make intelligent decisions.

Inspiration from nature, Innovation in machines!

Components of ANN:

Inputs ($x_1, x_2, \ldots, x_n$): Input features.

Weights ($w_1, w_2, \ldots, w_n$): Importance of each input.

Bias (b): Adjusts the activation threshold.

Summation (Σ): Computes the weighted sum.

Activation Function (f): Determines whether the neuron should produce an output.

Output (y): Final prediction passed to the next layer.

Mathematical Equation

$$z = \sum_{i=1}^{n} w_i x_i + b$$

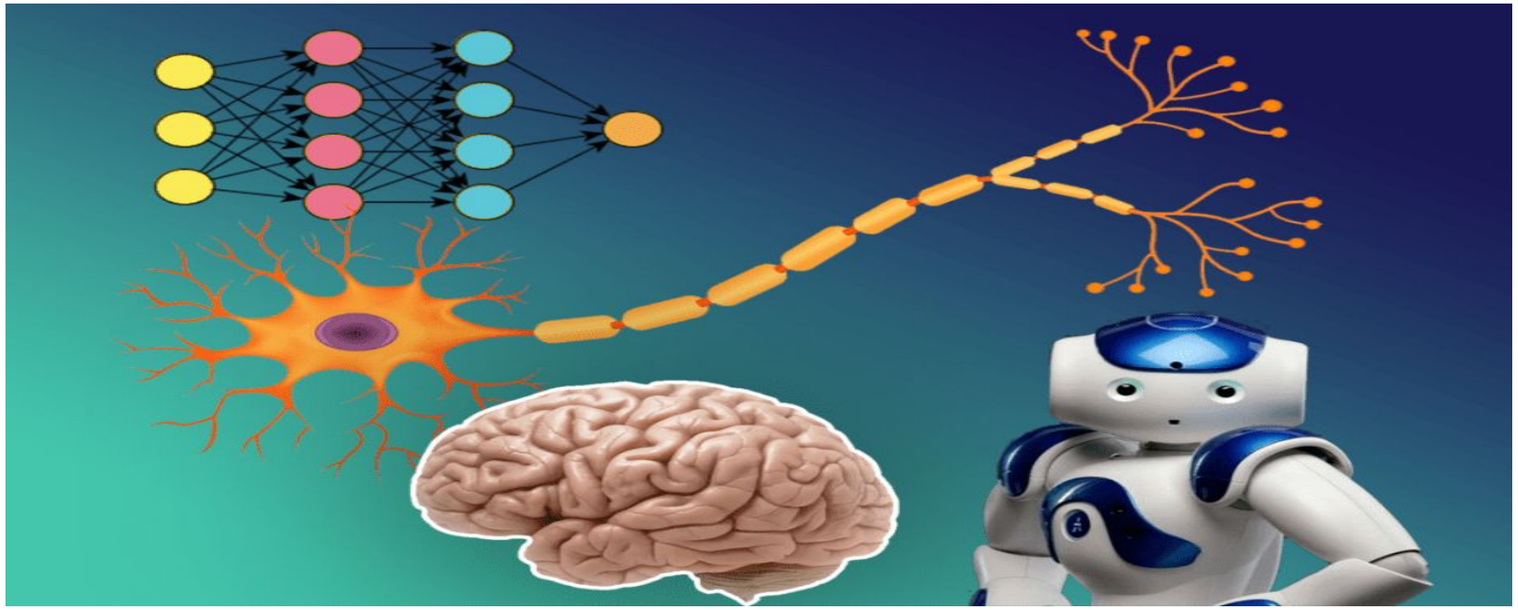
$$y = f(z)$$

where:

- x = Input
- w = Weight
- b = Bias
- z = Weighted sum
- f(z) = Activation function
- y = Output

UNIT-I

Artificial Neural Network



ARTIFICIAL NEURAL NETWORK-PERCEPTRON

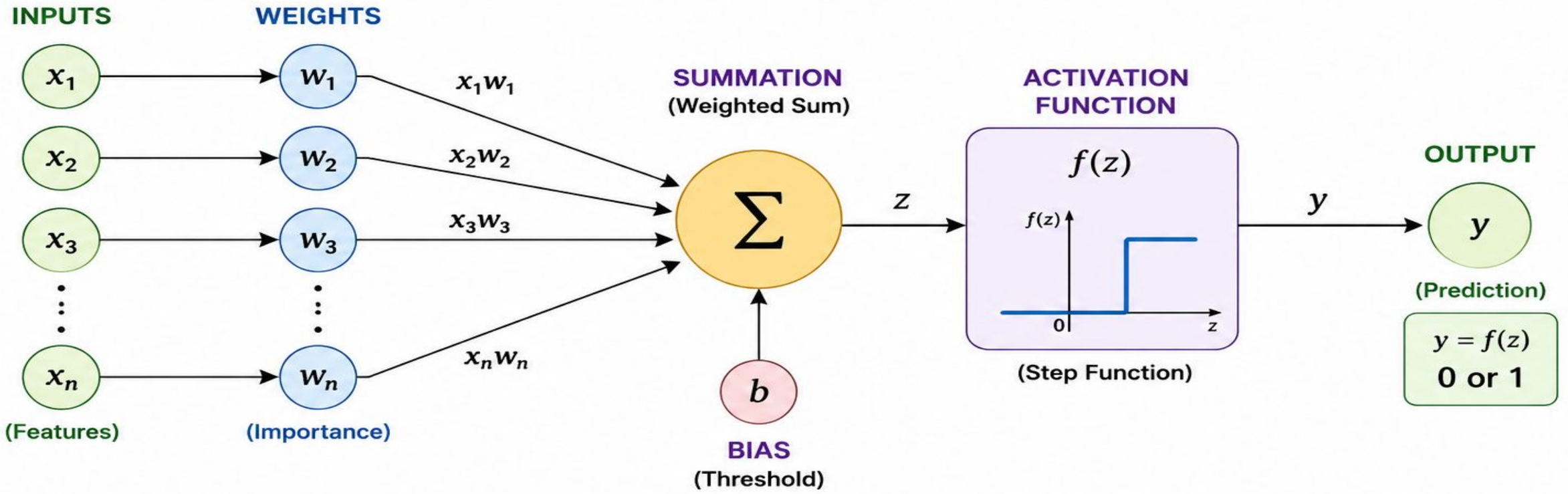
- A **Perceptron** is **an Artificial Neuron**
- **Frank Rosenblatt (1928 – 1971)** was an **American psychologist** notable in the field of **Artificial Intelligence**.
- In 1957 , he **"invented"** a **Perceptron program**, on **an IBM 704 computer** at **Cornell Aeronautical Laboratory**.
- **The Neurons receive input** from **our senses** by **electrical signals** and **store information** to **make decisions** based on **previous input.**
- So, A **Perceptron Unit** is **used to build** the **ANN System.**

A **Perceptron** is the **simplest type of Artificial Neural Network (ANN)** and the **fundamental building block of Deep Learning.**

A perceptron is a **single artificial neuron** that **receives multiple inputs, processes them using weights and a bias, applies an activation function, and produces an output.**

PERCEPTRON DIAGRAM

(Single Layer Neural Network / Artificial Neuron)



MATHEMATICAL MODEL

$$z = \sum_{i=1}^{n} w_i x_i + b$$

$$y = f(z)$$

x_i	=	Input signals
w_i	=	Weights of inputs
b	=	Bias (Threshold)
z	=	Net input (Weighted Sum)
$f(z)$	=	Activation Function (Step Function)
y	=	Output (0 or 1)

STEP FUNCTION

$$f(z) = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{if } z < 0 \end{cases}$$

WORKING

1. Multiply each input with its weight.
2. Add all the weighted inputs and bias to get z.
3. Apply activation function $f(z)$.
4. Produce output y (0 or 1).

PERCEPTRON LEARNING ALGORITHM

Frank Rosenblatt suggested this algorithm:

- **Step-1: Set a threshold value**
- **Step-2: Multiply all inputs with its weights**
- **Step-3: Sum all the results**
- **Step-4: Activate the output**

EXAMPLE

- Imagine a perceptron (in your brain).
- The perceptron tries to decide if you should go to a concert.
- Is the artist good?
- Is the weather good?
- What weights should these facts have?

Criteria	Input	Weight
Artists is Good	**x1** = 0 or 1	**w1** = 0.7
Weather is Good	**x2** = 0 or 1	**w2** = 0.6
Friend will Come	**x3** = 0 or 1	**w3** = 0.5
Food is Served	**x4** = 0 or 1	**w4** = 0.3
Alcohol is Served	**x5** = 0 or 1	**w5** = 0.4

EXAMPLE

1. Set a threshold value:

- Threshold = 1.5

2. Multiply all inputs with its weights:

- x1 * w1 = 1 * 0.7 = 0.7
- x2 * w2 = 0 * 0.6 = 0
- x3 * w3 = 1 * 0.5 = 0.5
- x4 * w4 = 0 * 0.3 = 0
- x5 * w5 = 1 * 0.4 = 0.4

3. Sum all the results:

- 0.7 + 0 + 0.5 + 0 + 0.4 = 1.6 (The Weighted Sum)

4. Activate the Output:

- Return true if the sum > 1.5 ("Yes I will go to the Concert")



Perceptron Terminology

Perceptron Terminology:

- **1. Inputs :** Here Inputs called **Nodes**

  In the above example, the node values are: **1, 0, 1, 0, 1**

  The binary **input values** (0 or 1) can be interpreted as (no or yes) or (false or true).

- **2. Node Weights:** Weights shows the **strength of each node**.

  In the above example, the Weights are **0.7, 0.6, 0.5, 0.3, 0.4**

- **3. Activation Function:** The **activation functions maps the result (the weighted sum)** into a **required value like 0 or 1.**

- In the **above Example,** the **activation function** is simple: **(sum > 1.5)**

- The **binary output (1 or 0)** can be **interpreted as (yes or no) or (true or false).**

- **Note:** It is obvious that a ***decision is NOT made by one neuron alone.***

- **Threshold Value:** We have to define **a reliable threshold** for the **classifier** that **clearly indicates the split between the two classes.**

Perceptron Terminology

Bias: **Bias** is a **trainable parameter added to the weighted sum of inputs** in **an artificial neuron**.

- It **shifts the activation function** and **allows the neuron to produce a non-zero output** even **when all input values are zero.**
- **Bias increases the flexibility of the neural network**, enabling it to learn more complex patterns and make more accurate predictions. Mathematically, a neuron computes its output as:

$$z = \sum_{i=1}^{n} w_i x_i + b$$

Example : Student Marks

Suppose a student scores

Exam Marks = 34
Pass Mark = 35

Without grace marks

34 < 35
Fail

Teacher gives

Grace Marks = 2

Now

34 + 2 = 36
Pass

Here,

Exam Marks → Weighted Sum

Grace Marks → Bias

Exactly the same happens **inside a neural network.**

Bias adjusts the weighted sum before making the decision



REPRESENTATION OF POWER OF PERCEPTRON

- As we can see, **it calculates a weighted sum of its inputs** and **thresholds it with a step function**.
- A **single perceptron** can be **used to represent many Boolean functions.**
- The Boolean Functions , **AND, OR, XOR, NAND, NOR** can be solved the Perceptron.
- **Geometrically,** this means **the perceptron can separate its input space** with a **Hyper Plane**.
- That's where the notion that a perceptron can **only separate linearly separable problems** .

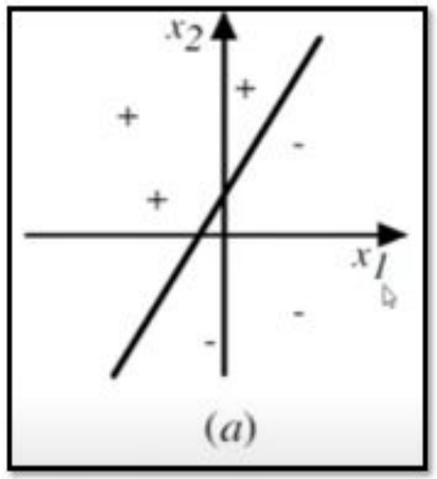
Types of Perceptron's:

- **1. Single Layer Perceptron Model**
- **2. Multi Layer Perceptron Model**

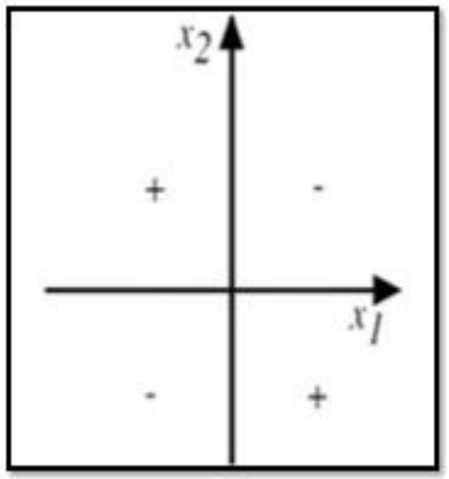
- **1. Single Layer Perceptron Model:** The **main objective** of the single-layer perceptron model is **to analyze the linearly separable objects** with **binary outcomes**.
- Single Layer perceptron , **can only solve Linearly Separable Problems.**



- Some of the **Boolean Functions cannot represent the Single Perceptron,** Such as **XOR function** is **not linearly separable,** it really is impossible for **a single hyper plane to separate it**.



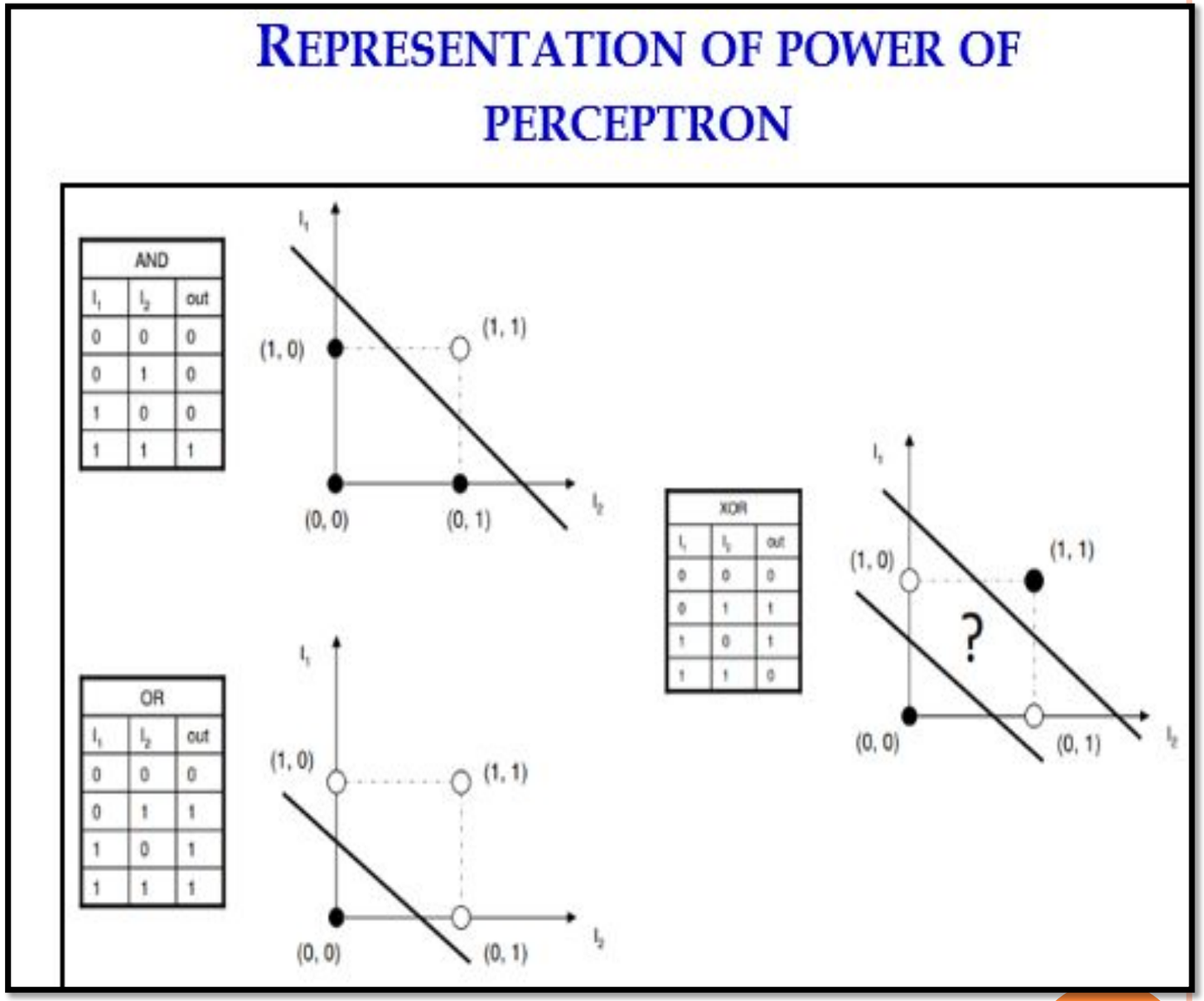
Linearly Separable



Not Linearly Separable

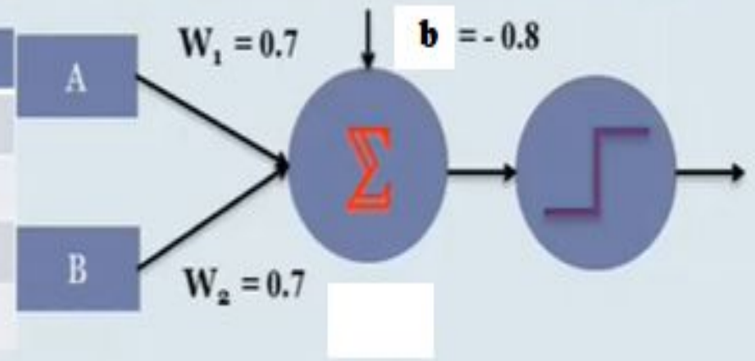
- **2. Multi Layer Perceptron Model:** Like a **single-layer perceptron model**, a multi-layer perceptron model also has the same model structure but has a **greater number of hidden layers.**
- The **multi-layer perceptron model** is also known as the **Back Propagation Algorithm.**
- Hence, **a multi-layered perceptron model** has considered as **artificial neural networks** having **various layers** in which **activation function does not remain linear**, similar to a single layer perceptron model.

- **Instead of linear,** activation function can be executed as **sigmoid, TanH, ReLU, etc., for deployment.**
- A multi-layer perceptron model has **greater processing power** and can **process linear and non-linear patterns**.
- Further, it can also **implement logic gates** such as **AND, OR, XOR, NAND, NOT, XNOR, NOR.**



- **Example: Representation of AND functions**

A	B	A ^ B
0	0	0
0	1	0
1	0	0
1	1	1



If A=0 and B=0

$b + W_1 * A + W_2 * B = -0.8 + 0*0.7 + 0*0.7 = -0.8.$

This is not greater than the threshold of 0, so the output = -1.

- If A= 0 and B=1
- $b + W_1 * A + W_2 * B = -0.8 + 0*0.7 + 1*0.7 = -0.1.$
- This is not greater than the threshold value of 0, so the output = -1.
- If A= 1 and B=0
- $b + W_1 * A + W_2 * B = -0.8 + 1*0.7 + 0*0.7 = -0.1.$
- This is not greater than the threshold value of 0, so the output = -1.

- If A= 1 and B=1

$$b + W_1 * A + W_2 * B$$
$$= -0.8 + 1 * 0.7 + 1*0.7 = 0.6$$

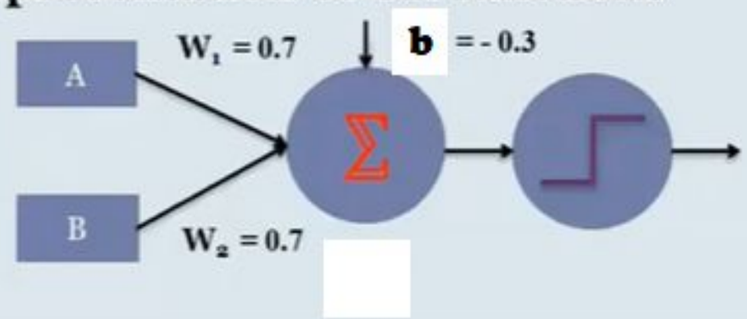
- This is not greater than the threshold value of 0, so the output = +1.

So , the Final weights, b= -0.8 , W1 = 0.7 , W2 = 0.7

By using these weights, All Classified Correctly.

- **Example: Representation of OR functions**

A	B	A v B
0	0	0
0	1	1
1	0	1
1	1	1



If A=0 OR B=0

$b + W_1 * A + W_2 * B = -0.3 + 0*0.7 + 0*0.7 = -0.3.$

This is not greater than the threshold of 0, so the output = -1.

- If A= 0 OR B=1
- $b + W_1 * A + W_2 * B = -0.3 + 0*0.7 + 1*0.7 = 0.4$
- This is not greater than the threshold value of 0, so the output = +1.
- If A= 1 OR B=0
- $b + W_1 * A + W_2 * B = -0.3 + 1*0.7 + 0*0.7 = 0.4$
- This is not greater than the threshold value of 0, so the output = +1.

- If A= 1 & B=1

$$b + W_1 * A + W_2 * B$$
$$= -0.3 + 1 * 0.7 + 1 * 0.7 = 1.1$$

- This is not greater than the threshold value of 0, so the output = +1.
- Drawback of perceptron
- The perceptron rule finds a successful weight vector when the training examples are linearly separable, it can fail to converge if the examples are not linearly separable

PERCEPTRON LEARNING RULE

- This **rule revises** the **weight, wi** associated with the **input xi** according to the rule.

$$w_i \leftarrow w_i + \Delta w_i$$

where

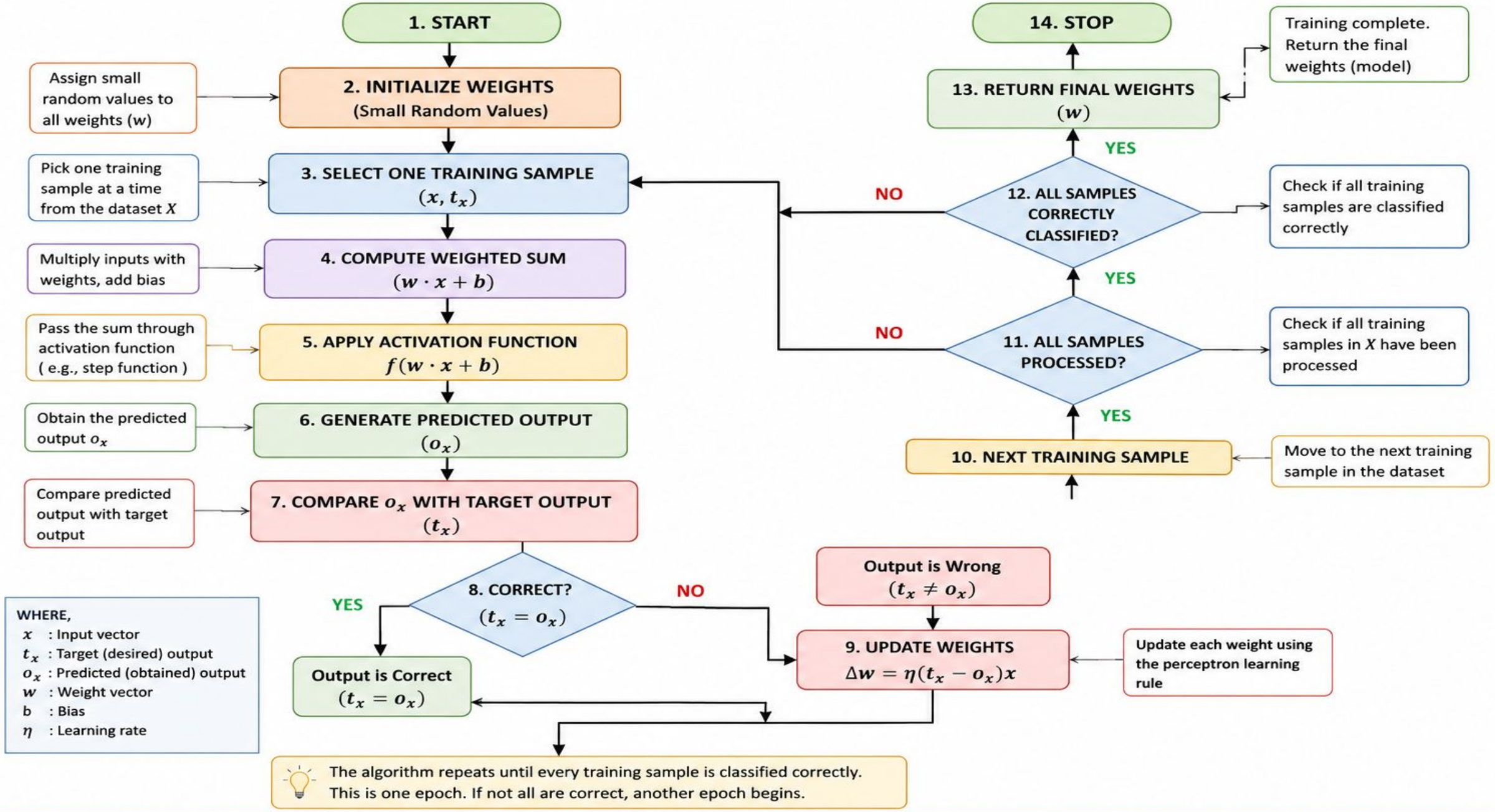
$$\Delta w_i = \eta(t - o)x_i$$

- Here, t = Targeted Outpu
- o = Actual Output
- **Learning Rate:** The learning rate is a **hyperparameter** that **controls how much to change the model in response to the estimated error each time** the **model weights** are **updated.**

ALGORITHM

Perceptron_training_rule *(X, η)*
initialize ***w*** *(wi ← an initial (small) random value)*
repeat
    for each training instance ***(x, tx)*** $\in X$
        compute the real output ***ox*** *= Activation(Summation(****w.x****))*
        if ***(tx ≠ ox)***
            for each wi
                $wi \leftarrow wi + \Delta wi$
                $\Delta wi \leftarrow \eta\ (tx - ox)xi$
            end for
        end if
    end for
until all the training instances in X are correctly classified
return ***w***

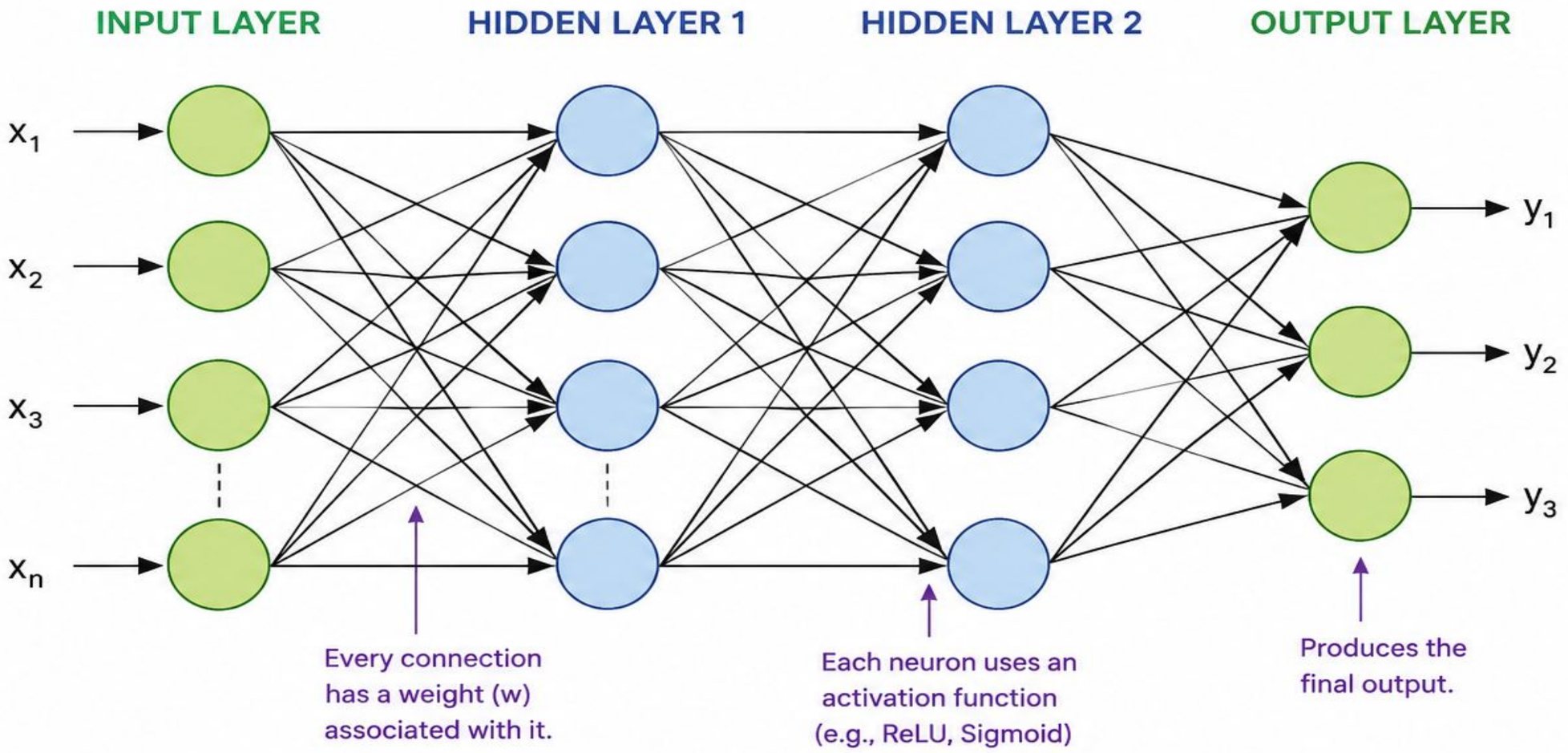
PERCEPTRON TRAINING ALGORITHM (FLOWCHART)



Layers in ANN

- Artificial Neural Networks are **processing elements** either in the form of **algorithms or hardware devices modeled.**
- Artificial Neural Network primarily consists of **Three Layers:**
  1. **Input Layer**
  2. **Hidden Layer**
  3. **Output Layer**
- **Each Layer** consists of **more Neurons**
- The **nodes of the Input Layer** can take **input data and perform operations on it** and **send the results** of these operations to **other neurons of the Hidden Layer.**
- The **hidden layer** sends **data to the Output Layer.**

ARTIFICIAL NEURAL NETWORK (ANN)

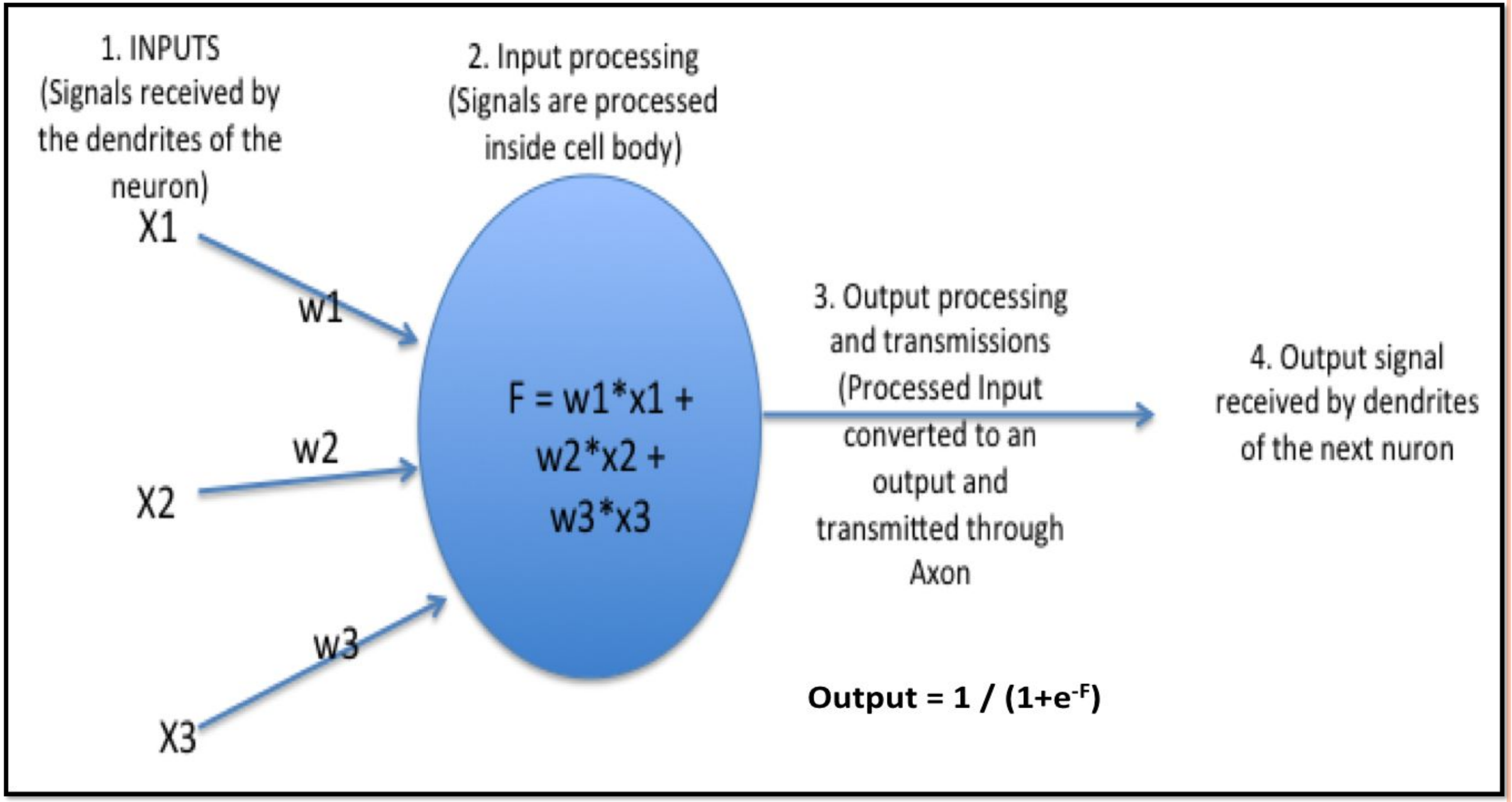


How It Works:

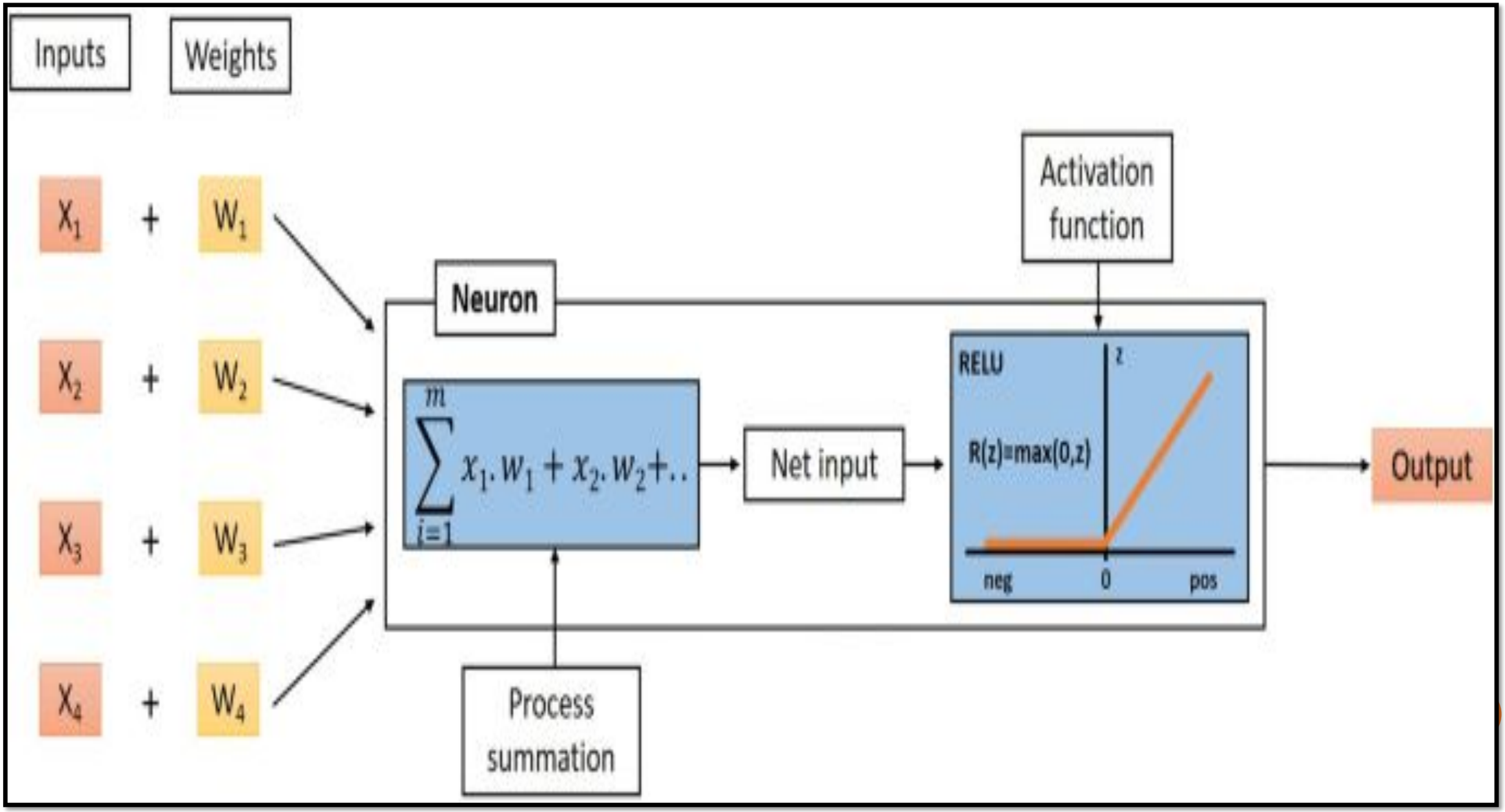
1. Inputs are fed into the input layer.
2. Each connection multiplies the input by a weight.
3. The neuron sums the weighted inputs, applies an activation function.
4. Output is passed to the next layer until the final output is produced.

Input Neuron Hidden Neuron Output Neuron Weighted Connection

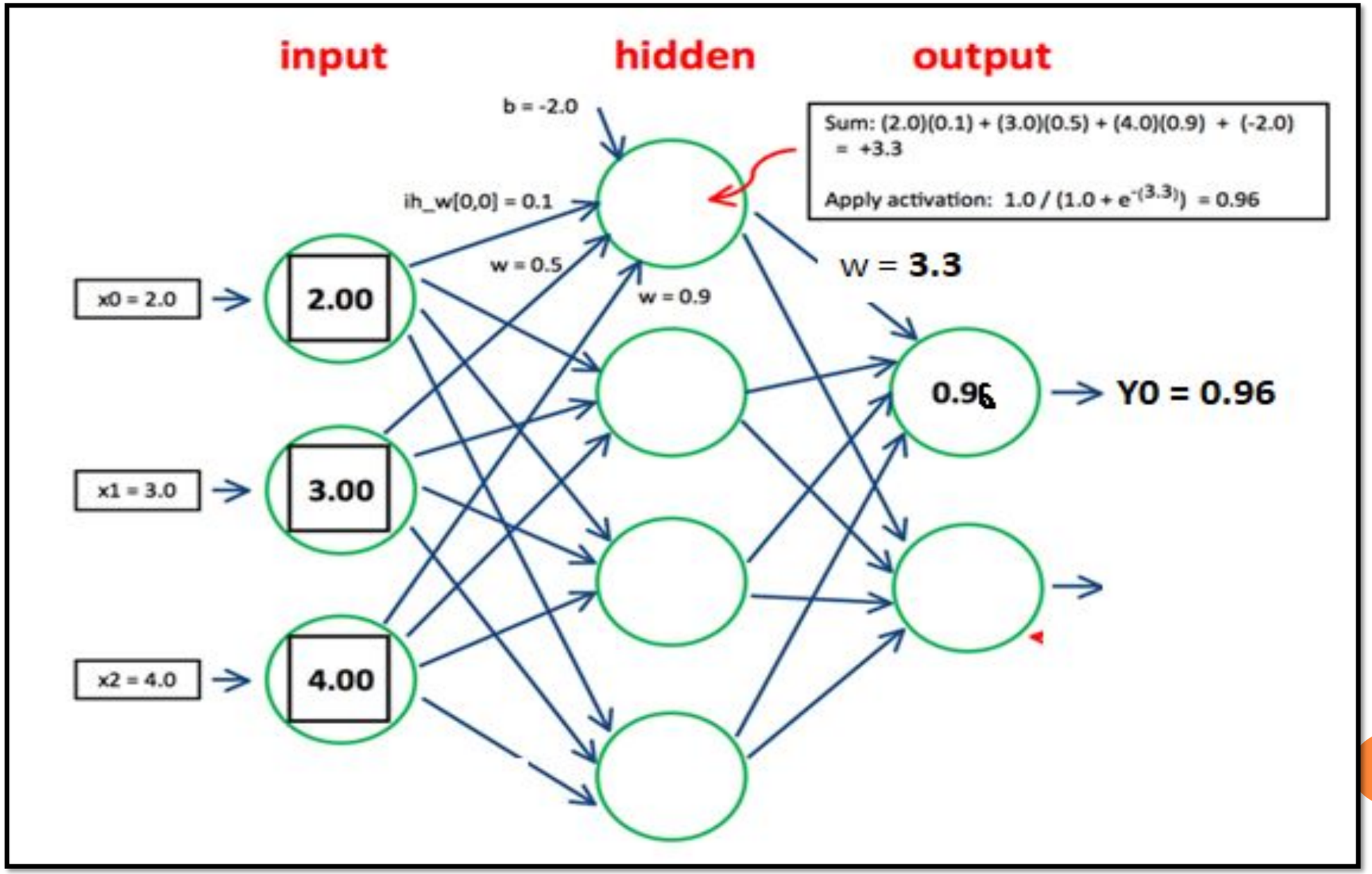
What happens inside the neuron



What happens inside the neuron



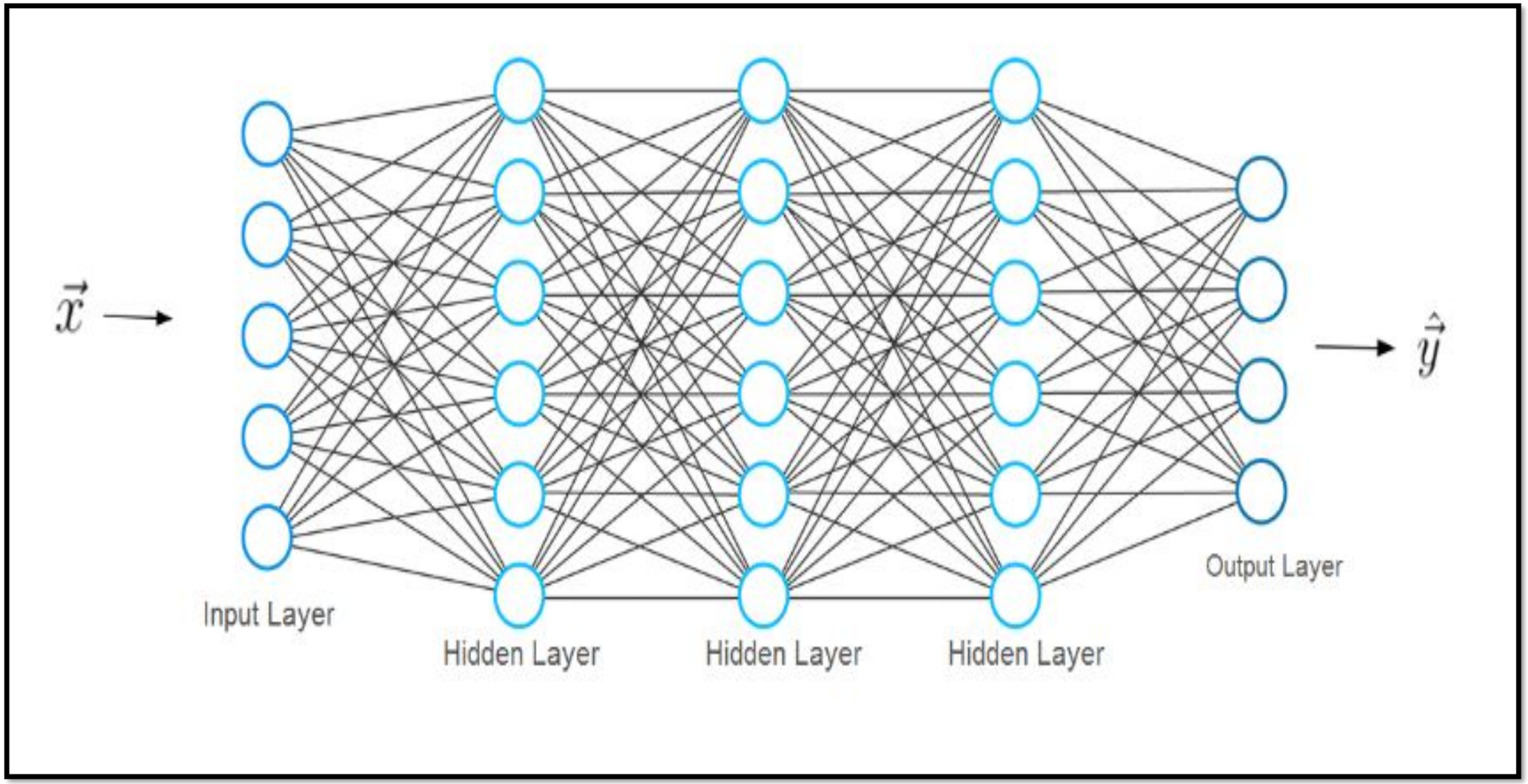
EXAMPLE


input
hidden
output
b = -2.0
Sum: (2.0)(0.1) + (3.0)(0.5) + (4.0)(0.9) + (-2.0)
= +3.3
Apply activation: 1.0 / (1.0 + e^-(3.3)) = 0.96
ih_w[0,0] = 0.1
w = 0.5
w = 0.9
W = 3.3
x0 = 2.0
2.00
x1 = 3.0
3.00
x2 = 4.0
4.00
0.96
Y0 = 0.96

Layered Connection of Neural Network

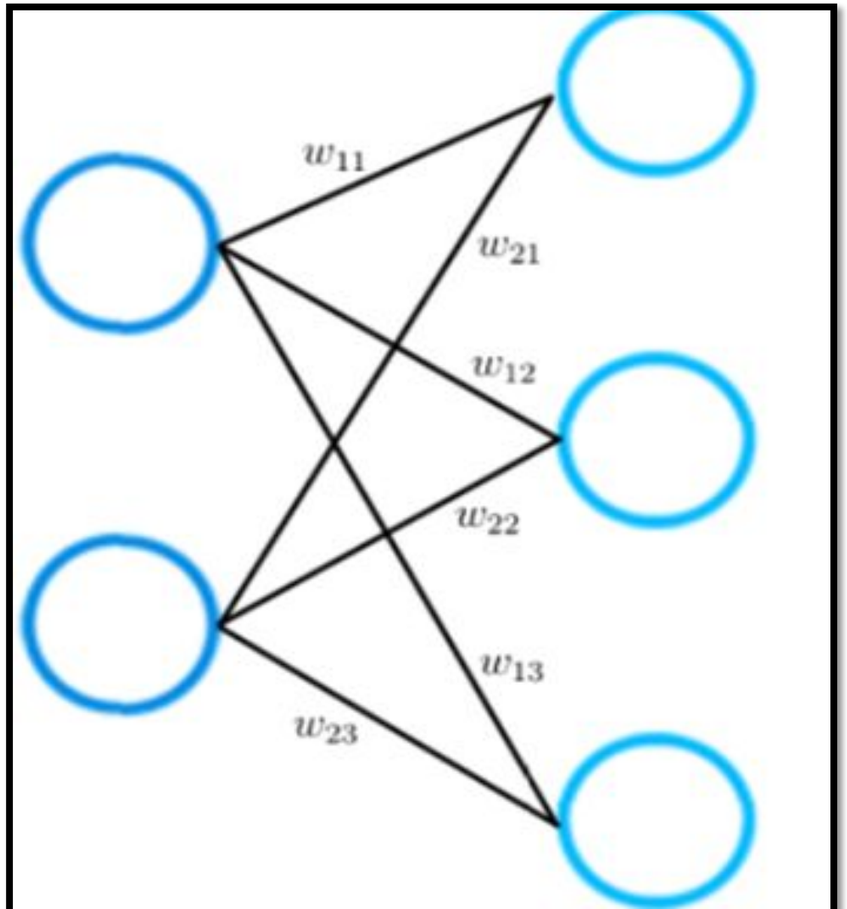
Three basic components are used in ANN are

1.Components **2. Learning** **3. Activation Function**



Explanation of Layers in ANN

The **input layer** has **two input neurons**, while **the output layer** consists of **three neurons**.



- Each **connection** between **two neurons** is represented by a **different weight w**. **The first value** of the indices stands for the **number of neurons in the layer** from **which the connections originate.**
- **The second value** for the number of the neurons in the layer to **which the connection leads.**
- All weights between **two neural network layers** can be represented **by a matrix** called the **weight matrix.**

Learning Process of NN

$$W = \begin{pmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \end{pmatrix}$$

A weight matrix.

In this particular **example**, the **number of rows** of the weight matrix corresponds to the **size of the input layer which is two** and the **number of columns** to **the size of the output layer which is three.**

For a given **input feature vector x**, the **neural network calculates** a **prediction vector**, which we call **here as h.**

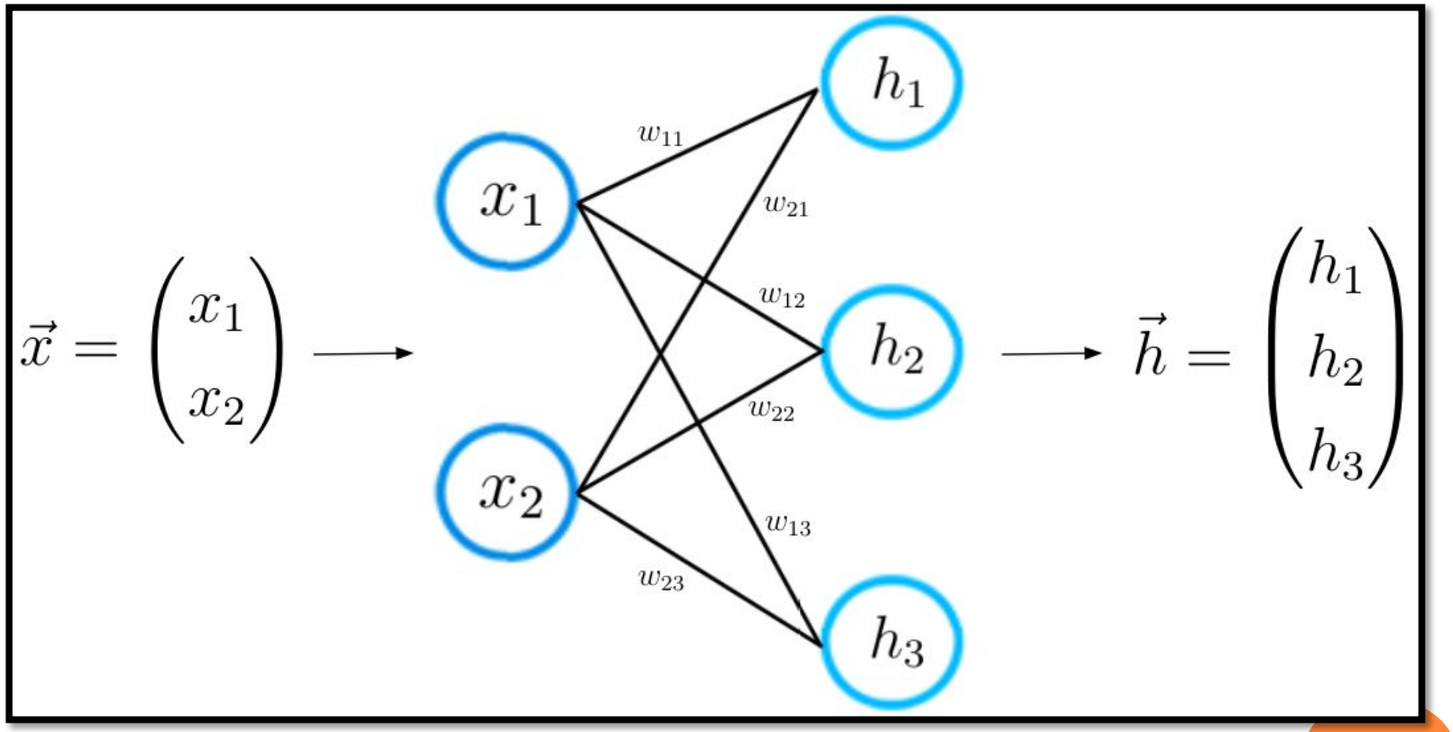


Figure: Forward Propagation

- The **final prediction vector h** is obtained by applying a so-called **activation function** to the **vector z**. In this case, the activation function is represented by the **letter Sigma.**
- Here **neuron i**s simply a representation of a **numeric value.**

$$\vec{x}^T \cdot W = \begin{pmatrix} x_1, & x_2 \end{pmatrix} \cdot \begin{pmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \end{pmatrix}$$

$$= \begin{pmatrix} x_1 w_{11} + x_2 w_{21}, & x_1 w_{12} + x_2 w_{22}, & x_1 w_{13} + x_2 w_{23} \end{pmatrix}$$

$$= \begin{pmatrix} z_1, z_2, z_3 \end{pmatrix} = \vec{z}, \quad \vec{h} = \sigma(\vec{z})$$

- This step is also referred to as the **forward propagation.** With the **input vector x and the weight matrix W** connecting the two neuron layers, **we compute the dot product** between the **vector x and the matrix W.** The result of this dot product is again a vector, which **we call it as Z**

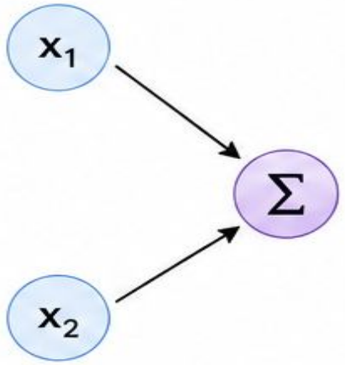
Activation Functions in ANN

An **Activation function** is a **mathematical function** **applied to the output of a neuron** after **calculating the weighted sum of inputs**. It determines whether a **neuron should be activated** and **pass information to the next layer**.



WORKING OF AN ACTIVATION FUNCTION

STEP 1
Receive Inputs

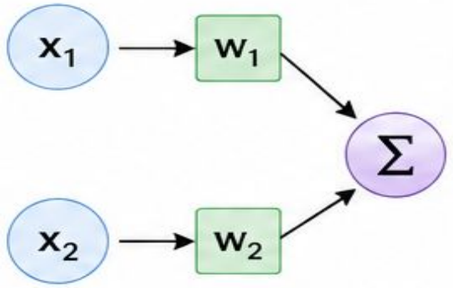


Example:

$x_1 = 2$

$x_2 = 3$

STEP 2
Multiply by Weights



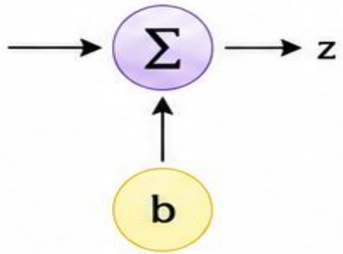
Example:

$w_1 = 0.5$

$w_2 = 0.8$

STEP 3
Add Bias

$$z = w_1x_1 + w_2x_2 + b$$



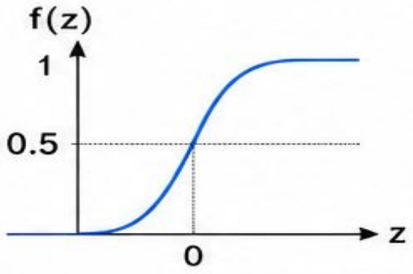
Example:

$z = (2 \times 0.5) + (3 \times 0.8) + 1$

$z = 4.4$

STEP 4
Apply Activation Function

$$\text{Output} = f(z)$$

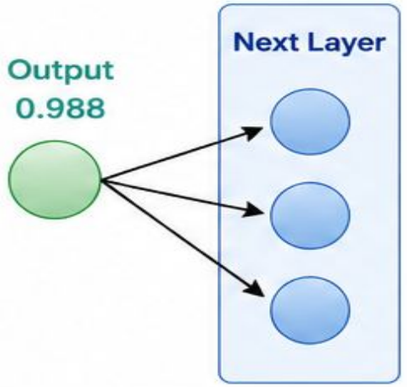


Example (Using Sigmoid):

$\text{Output} = f(4.4)$

$\text{Output} \approx 0.988$

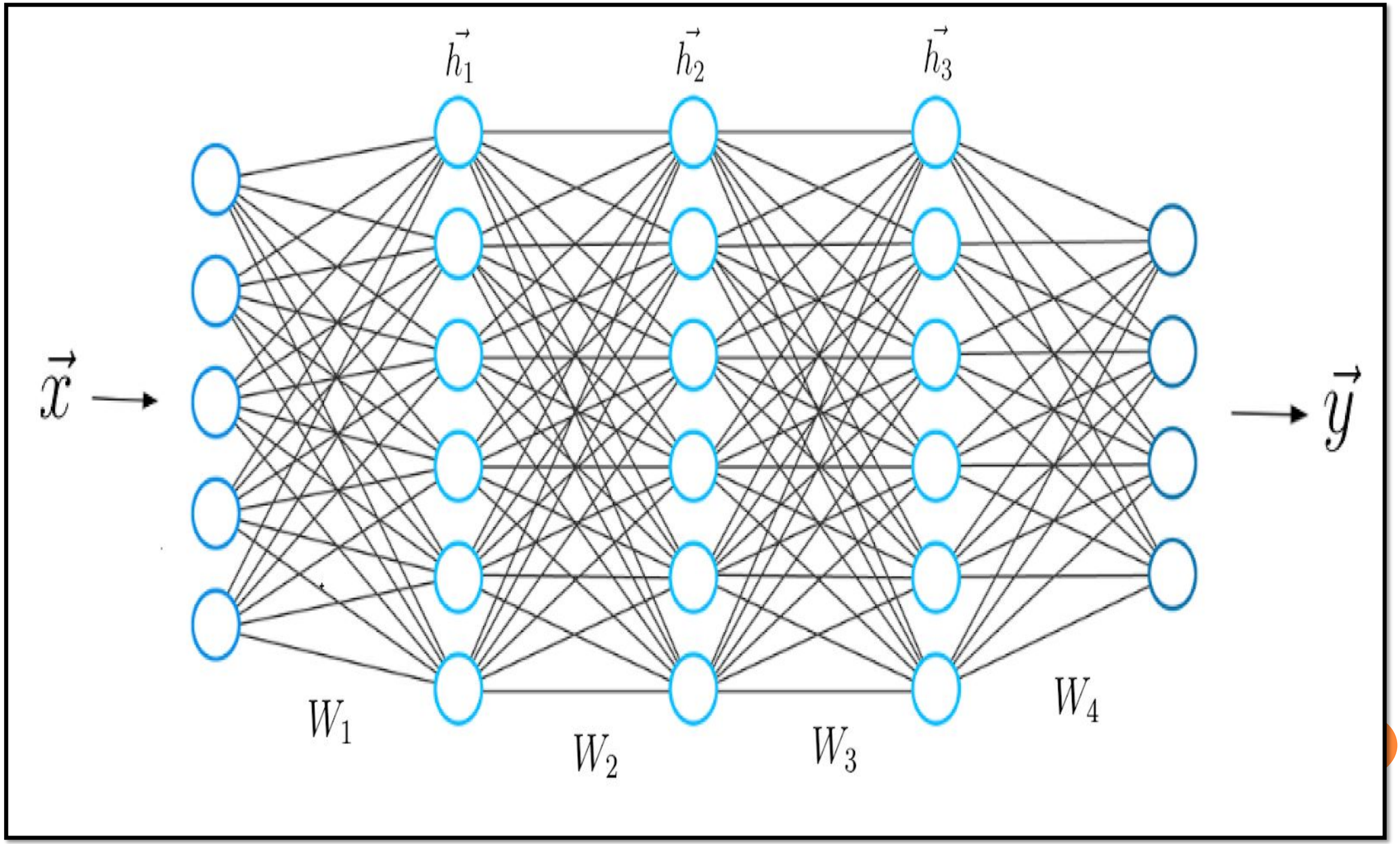
STEP 5
Send Output to the Next Layer



SUMMARY FLOW

Inputs (x_1, x_2) → Multiply by Weights (w_1, w_2) → Add Bias (b) → Apply Activation Function f(z) → Output (Send to Next Layer)

x_1, x_2 = Inputs	w_1, w_2 = Weights	b = Bias	z = Weighted Sum	f(z) = Activation Function Output


$\vec{h}_1$
$\vec{h}_2$
$\vec{h}_3$
$\vec{x}$
$\vec{y}$
W_1
W_2
W_3
W_4

- we can extend our knowledge to a deeper architecture that **consists of 5 layers.**
- **h1 i**s now considered as the i**nput** for the upcoming **third layer.** The whole procedure from before is repeated until we obtain the **final output y:**

$$\vec{h_1} = \sigma(\vec{x}^T \cdot W_1)$$
$$\vec{h_2} = \sigma(\vec{h_1} \cdot W_2)$$
$$\vec{h_3} = \sigma(\vec{h_2} \cdot W_3)$$
$$\vec{y} = \sigma(\vec{h_3} \cdot W_4)$$

Real time Applications of Perceptron's

Any Digital Logic Circuit can be implemented by using Perceptron.

